Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №2.

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Розробка графічного редактора об’єктів »

Виконала:

студентка групи ІМ-53
Кіосова Софія Олександрівна
Номер в списку - 13

Перевірив:

Рекечинський Д. О

Київ 2026

Завдання

Вимоги

2. Усі методи-обробники повідомлень, зокрема, і метод OnNotify, повинні бути функціями-членами деякого класу (класів). 3. Для вибору типу об’єкту в графічному редакторі Lab3 повинно бути вікно Toolbar з кнопками відповідно типам об'єктів. Кнопки дублюють підпункти меню "Об’єкти". Кнопки мають бути з підказками (tooltips). Кнопки у Toolbar повинні бути не звичайними з текстовим підписом, а кнопкою з іконкою (приклад можете побачити на Рис. 3.1). Іконки слід підбирати таким чином, щоб можна було здогадатись, за який інструмент відповідає ця кнопка.

Завдання згідно варіанту

4. Для вибору варіанту використовується значення Ж = Жлаб2 + 1, де Жлаб2 – номер студента в журналі, який використовувався для попередньої лабораторної роботи №2. 5. Повністю повторюєте вимоги лабораторної роботи №2 від пункту 4 до пункту 7 із новим значенням Ж. – у моєму випадку нове Ж це 14, відповідно нова робота має вигляд такий:

1. Для вибору типу об’єкта в графічному редакторі Lab2 повинно бути меню "Об’єкти" з чотирма підпунктами. Меню "Об’єкти" повинно бути праворуч меню "Файл" та ліворуч меню "Довідка". Підпункти меню "Об’єкти" містять назви українською мовою геометричних форм. Геометричні форми згідно варіанту завдання
2. Масив вказівників для динамічних об’єктів типу Shape N=114 (статичний Ж mod 3 = 2)
3. Гумовий" слід при вводі об’єктів суцільна лінія синього кольору для варіантів (Ж mod 4 = 2)
4. 6.1. Прямокутник (Увід- по двом протилежним кутам для варіантів (Ж mod 2 = 0) (Відображення - чорний контур прямокутника без заповнення для (Ж mod 5 = 3 або 4)) (Кольори заповнення - блакитний для (Ж mod 6 = 2))
5. Еліпс (Увід - від центру до одного з кутів охоплюючого прямокутника для варіантів (Ж mod 2 = 0)) (Відображення - чорний контур з кольоровим заповненням для (Ж mod 5 = 3 або 4)) (Кольори заповнення - світло-зелений для (Ж mod 6 = 2))
6. Позначка поточного типу об’єкту, що вводиться в меню (метод OnInitMenuPopup) для варіантів (Ж mod 2 = 0)

Структура проекту + код

- main.py – головний файл, точка входу у програму

```
import tkinter as tk
from window import Window


if __name__ == "__main__":
    root = tk.Tk()
    root.geometry("800x600")
```

```
    app = Window(root)
    root.mainloop()
```

- editor.py – Логіка малювання

```
from shapes import Shape, Point, Line, Rectangle, Ellipse


class Editor():
    def __init__(self, canvas):
        self.canvas = canvas
        self.MAX_SHAPES = 114
        self.shapes = []
        self.current_shape_type = "Rectangle"

        self.start_x = 0
        self.start_y = 0
        self.rubber_line_id = None
        self.is_drawing = False

        self.canvas.bind('<ButtonPress-1>', self.on_press)
        self.canvas.bind('<B1-Motion>', self.on_drag)
        self.canvas.bind('<ButtonRelease-1>', self.on_release)

    def set_shape_type(self, shape_type):
        self.current_shape_type = shape_type
        self.is_drawing = False
        self.start_x = 0
        self.start_y = 0
        if self.rubber_line_id:
            self.canvas.delete(self.rubber_line_id)
            self.rubber_line_id = None
```

```
    def on_press(self, event):
        self.is_drawing = True
        self.start_x = event.x
        self.start_y = event.y

    def on_drag(self, event):
        if not self.is_drawing:
            return
        if self.rubber_line_id:
            self.canvas.delete(self.rubber_line_id)

        temp_shape = None
        if self.current_shape_type == "Point":
            temp_shape = Point(self.start_x, self.start_y,
event.x, event.y)
        elif self.current_shape_type == "Line":
            temp_shape = Line(self.start_x, self.start_y,
event.x, event.y)
        elif self.current_shape_type == "Rectangle":
            temp_shape = Rectangle(self.start_x, self.start_y,
event.x, event.y)
        elif self.current_shape_type == "Ellipse":
            temp_shape = Ellipse(self.start_x, self.start_y,
event.x, event.y)

        if temp_shape:
            self.rubber_line_id =
temp_shape.draw_rubber_band(self.canvas)

        self.rubber_line_id = Shape.draw_rubber_band(
```

```
            self.canvas, self.start_x, self.start_y, event.x,
event.y
        )


    def on_release(self, event):
        if not self.is_drawing:
            return
        self.is_drawing = False
        if self.rubber_line_id:
            self.canvas.delete(self.rubber_line_id)
            self.rubber_line_id = None


        shape_obj = None
        if len(self.shapes) >= self.MAX_SHAPES:
            print(f"Досягнуто ліміту об'єктів
({self.MAX_SHAPES})!")
            return


        if self.current_shape_type == "Point":
            shape_obj = Point(self.start_x, self.start_y)
        elif self.current_shape_type == "Line":
            shape_obj = Line(self.start_x, self.start_y, event.x,
event.y)
        elif self.current_shape_type == "Rectangle":
            shape_obj = Rectangle(self.start_x, self.start_y,
event.x, event.y)
        elif self.current_shape_type == "Ellipse":
            shape_obj = Ellipse(self.start_x, self.start_y,
event.x, event.y)


        if shape_obj:
```

```
            shape_obj.draw(self.canvas)
            self.shapes.append(shape_obj)
            print(f"Додано об'єкт {self.current_shape_type}.
Всього у масиві: {len(self.shapes)}/{self.MAX_SHAPES}")
```

- shapes.py – Модель даних

```
from abc import ABC, abstractmethod

class Shape(ABC):
    def __init__(self, x1, y1, x2=0, y2=0):
        self.x1 = x1
        self.y1 = y1
        self.x2 = x2
        self.y2 = y2

    @abstractmethod
    def draw(self, canvas):
        pass

    @abstractmethod
    def draw_rubber_band(self, canvas):
        pass

class Point(Shape):
    def draw(self, canvas):
        return canvas.create_oval(self.x1 - 2, self.y1 - 2,
self.x1 + 2, self.y1 + 2, fill="black")
    def draw_rubber_band(self, canvas):
        return canvas.create_ovalself(self.x1 - 2, self.y1 - 2,
self.x1 + 2, self.y1 + 2, fill="red")

class Line(Shape):
    def draw(self, canvas):
        return canvas.create_line(self.x1, self.y1, self.x2,
self.y2, fill="blue")
    def draw_rubber_band(self, canvas):
```

```
            return canvas.create_line(self.x1, self.y1, self.x2,
    self.y2, fill="red", dash=(4, 2))

    class Rectangle(Shape):
        def draw(self, canvas):
            return canvas.create_rectangle(
                self.x1, self.y1,
                self.x2, self.y2,
                outline="black", fill="#8BF7FF"
            )
        def draw_rubber_band(self, canvas):
            return canvas.create_rectangle(
                self.x1, self.y1,
                self.x2, self.y2,
                outline="black", fill="#8BF7FF"
            )

    class Ellipse(Shape):
        def draw(self, canvas):
            rx = abs(self.x2 - self.x1)
            ry = abs(self.y2 - self.y1)
            left = self.x1 - rx
            top = self.y1 - ry
            right = self.x1 + rx
            bottom = self.y1 + ry
            return canvas.create_oval(
                left, top,
                right, bottom,
                outline="black", fill="#72FF8C"
            )

        def draw_rubber_band(self, canvas):
            rx = abs(self.x2 - self.x1)
            ry = abs(self.y2 - self.y1)
            left = self.x1 - rx
            top = self.y1 - ry
            right = self.x1 + rx
            bottom = self.y1 + ry
```

```
            return canvas.create_oval(
                left, top,
                right, bottom,
                outline="red", dash=(4, 2)
            )
```

- window.py – Побудова графічного інтерфейсу

```
import tkinter as tk
from editor import Editor
import os


class ToolTip():
    def __init__(self, widget, text):
        self.widget = widget
        self.text= text
        self.tip_window = None
        self.widget.bind("<Enter>", self.show_tip)
        self.widget.bind("<Leave>", self.hide_tip)


    def show_tip(self, event=None):
        if self.tip_window or not self.text:
            return
        x, y, _cx, _cy = self.widget.bbox("insert")
        x = x + self.widget.winfo_rootx() + 25
        y = y + self.widget.winfo_rooty() + 20
        self.tip_window = tw = tk.Toplevel(self.widget)
        tw.wm_overrideredirect(True)
        tw.wm_geometry(f"+{x}+{y}")
        label = tk.Label(
            tw, text=self.text, justify=tk.LEFT,
```

```
            background="#ffffe0", relief=tk.SOLID, borderwidth=1,
            font=("tahoma", "8", "normal")
        )
        label.pack(ipadx=1)

    def hide_tip(self, event=None):
        if self.tip_window:
            self.tip_window.destroy()
            self.tip_window = None

class Window():
    def __init__(self, root):
        self.root = root
        self.root.title("OOP_lab3 - [Прямокутник]")
        root.geometry("600x400")
        self.toolbar_frame = tk.Frame(root, bd=1,
relief=tk.RAISED)
        self.toolbar_frame.pack(side=tk.TOP, fill=tk.X)
        canvas = tk.Canvas(root, bg="lightgray")
        canvas.pack(fill=tk.BOTH, expand=True)
        self.editor = Editor(canvas)

        self.icons = {}
        self.buttons = {}

        self.create_menu()
        self.create_toolbar()
        self.select_shape("Rectangle")
```

```
    def create_menu(self):
        menubar = tk.Menu(self.root)
        self.root.config(menu=menubar)
        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Вихід",
command=self.root.quit)
        menubar.add_cascade(label="Файл", menu=file_menu)
        self.objects_menu = tk.Menu(menubar, tearoff=0)
        self.selected_type_var = tk.StringVar(value="Rectangle")

        shapes = [
            ("Крапка", "Point"),
            ("Лінія", "Line"),
            ("Прямокутник", "Rectangle"),
            ("Еліпс", "Ellipse")
        ]

        for label, shape_type in shapes:
            self.objects_menu.add_radiobutton(
                label=label,
                variable=self.selected_type_var,
                value=shape_type,
                command=lambda st=shape_type:
self.select_shape(st)
            )

        menubar.add_cascade(label="Об’єкти",
menu=self.objects_menu)
        help_menu = tk.Menu(menubar, tearoff=0)
```

```
        help_menu.add_command(label="Про програму",
command=self.show_about)
        menubar.add_cascade(label="Довідка", menu=help_menu)


    def create_toolbar(self):
        base_dir = os.path.dirname(os.path.abspath(__file__))
        tools = [
            ("Point", "Крапка", "dot.png"),
            ("Line", "Лінія", "line.png"),
            ("Rectangle", "Прямокутник", "rectangle.png"),
            ("Ellipse", "Еліпс", "elipse.png")
        ]

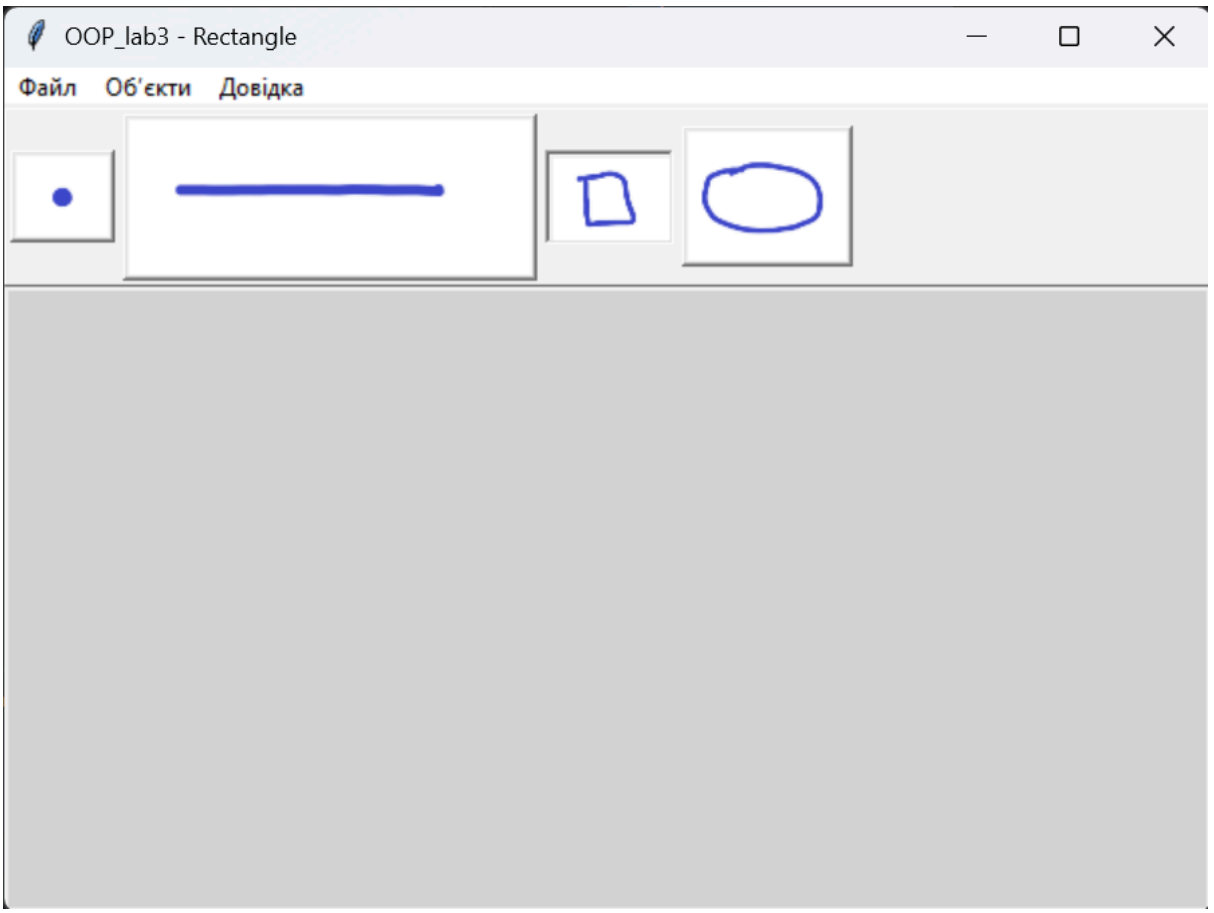

        for shape_type, tooltip_text, icon_file in tools:
            icon_path = os.path.join(base_dir, "icons",
icon_file)
            try:
                img = tk.PhotoImage(file=icon_path).subsample(3,
3)
                self.icons[shape_type] = img
                btn = tk.Button(
                    self.toolbar_frame, image=img,
relief=tk.RAISED,
                    command=lambda st=shape_type:
self.select_shape(st)
                )
            except Exception as e:
                print(f"Не вдалося завантажити іконку
{icon_path}: {e}")
                btn = tk.Button(
```

```
                    self.toolbar_frame, text=shape_type[0],
relief=tk.RAISED,
                    command=lambda st=shape_type:
self.select_shape(st)
                )
            btn.pack(side=tk.LEFT, padx=2, pady=2)
            ToolTip(btn, tooltip_text)
            self.buttons[shape_type] = btn

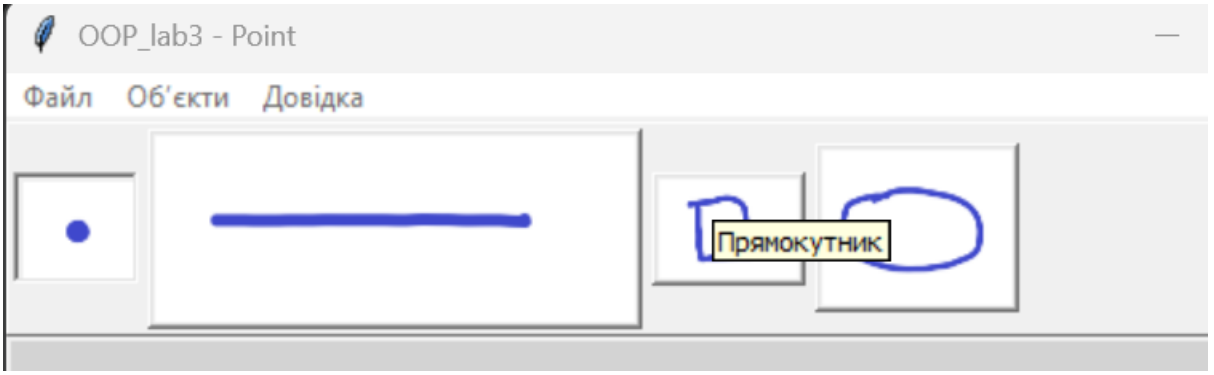
    def select_shape(self, type):
        self.editor.set_shape_type(type)
        self.selected_type_var.set(type)
        for st, btn in self.buttons.items():
            if st == type:
                btn.config(relief=tk.SUNKEN)
            else:
                btn.config(relief=tk.RAISED)
        self.root.title(f"OOP_lab3 - {type}")

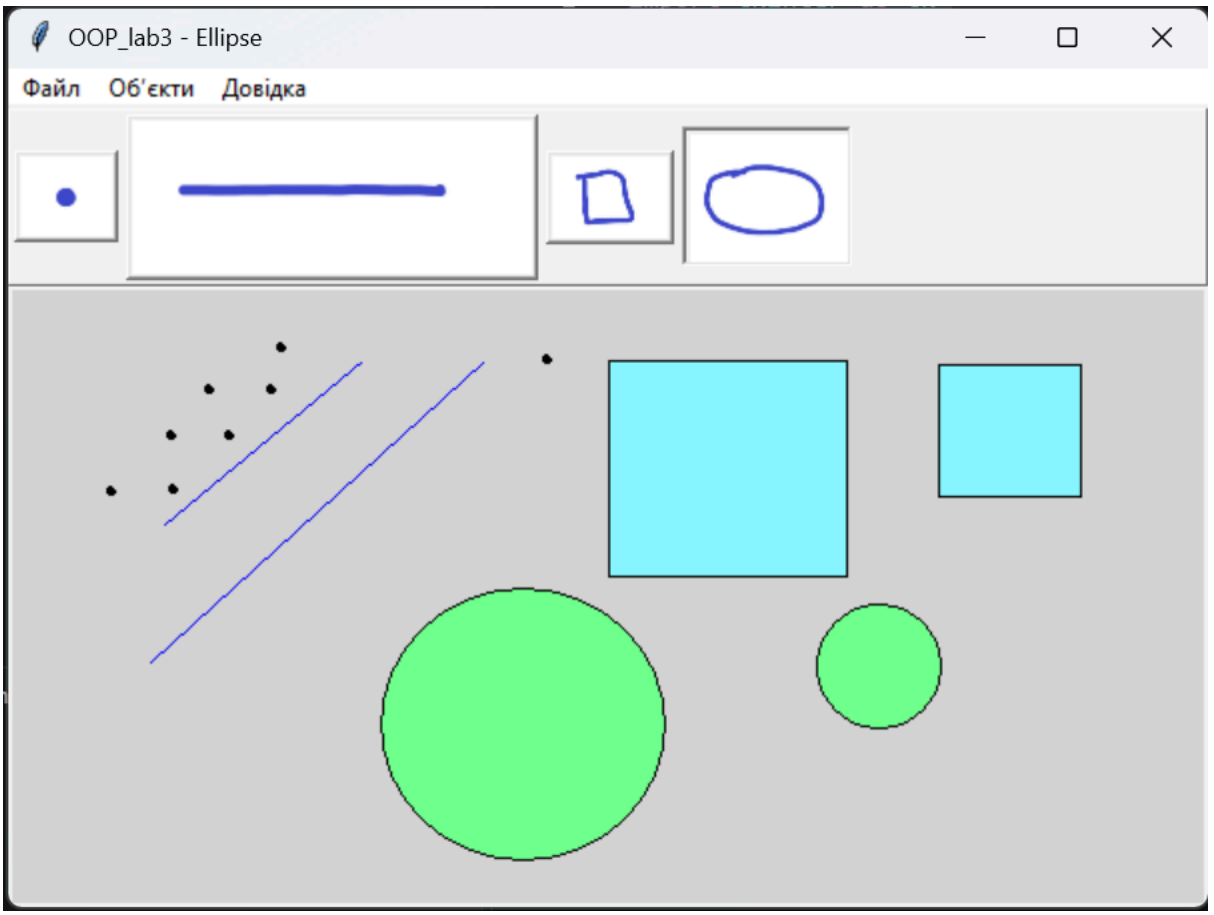

    def show_about(self):
        print("Лабораторна робота №3 з ООП")
```

Скріншоти

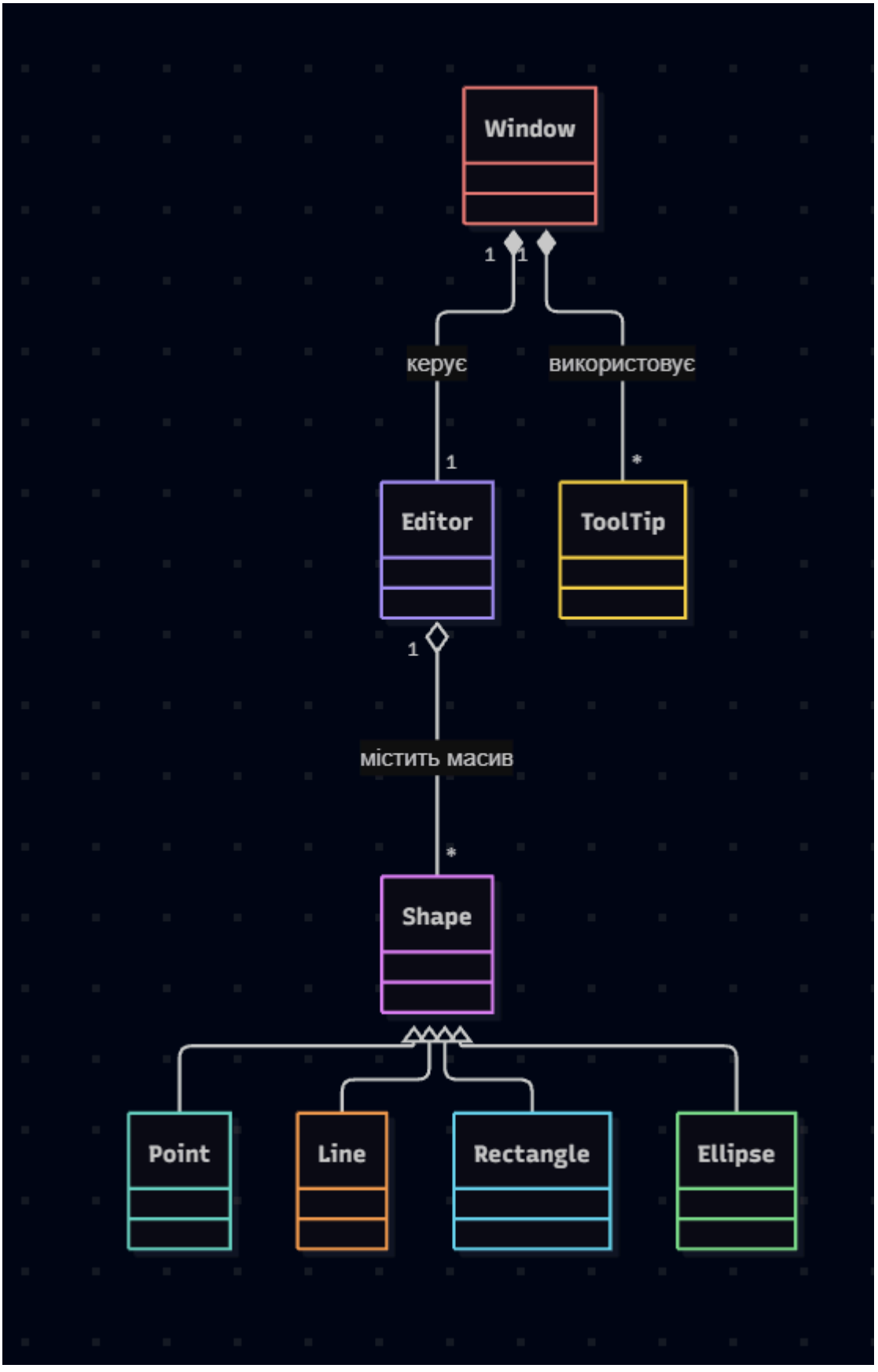

OOP_lab3 - Rectangle
Файл
Об'єкти
Довідка


Файл
Об'єкти
Довідка
Крапка


OOP_lab3 - Point
Файл
Об'єкти
Довідка
Прямокутник


OOP_lab3 - Ellipse
Файл
Об'єкти
Довідка

Діаграма


Window
1
1
керує
використовує
1
*
Editor
ToolTip
1
містить масив
*
Shape
Point
Line
Rectangle
Ellipse

Висновки: Виконавши 3 лабораторну работу, навчилась прив’язувати іконки до відповідних кнопок, забезпечувати відповідність пунктів меню і кнопок Toolbar, а також показ зображень. Крім того, було створено вивід підказок, закріплено роботу з абстрактними класами